Data Structures using C

Unit I

Data Structure:

- A data structure is a particular way of storing and organizing data in a computer. In other words, **the organised collection of data is known as data structure.**

Data structure = organised data + operations

- **Examples:** Arrays, Linked lists, Queues, Stacks, Trees, and Graphs
- Data structures are the building blocks of any program or the software.

(OR)

- A data structure is a storage that is used to store and organize data.
- It is way of arranging data on a computer so that it can be accessed and updated efficiently.
- A data structure is not only used for organizing the data.
- It is also used for processing, retrieving and storing data.

Example of Data Structure:

Need for Data Structure

- It gives different level of organization data.
- It tells how data can be stored and accessed in its elementary level.
- Provide operation on group of data, such as adding an item, looking up highest priority item.
- Provide a means to manage huge amount of data efficiently.
- Provide fast searching and sorting of data.

Goals of Data Structure

Data structure basically implements two complementary goals.

- **Correctness:** Data structure is designed such that it operates correctly for all kinds of input, which is based on the domain of interest.
- **Efficiency:** Data structure also needs to be efficient. It should process the data at high speed without utilizing much of the computer resources such as memory space.

Features of Data Structure

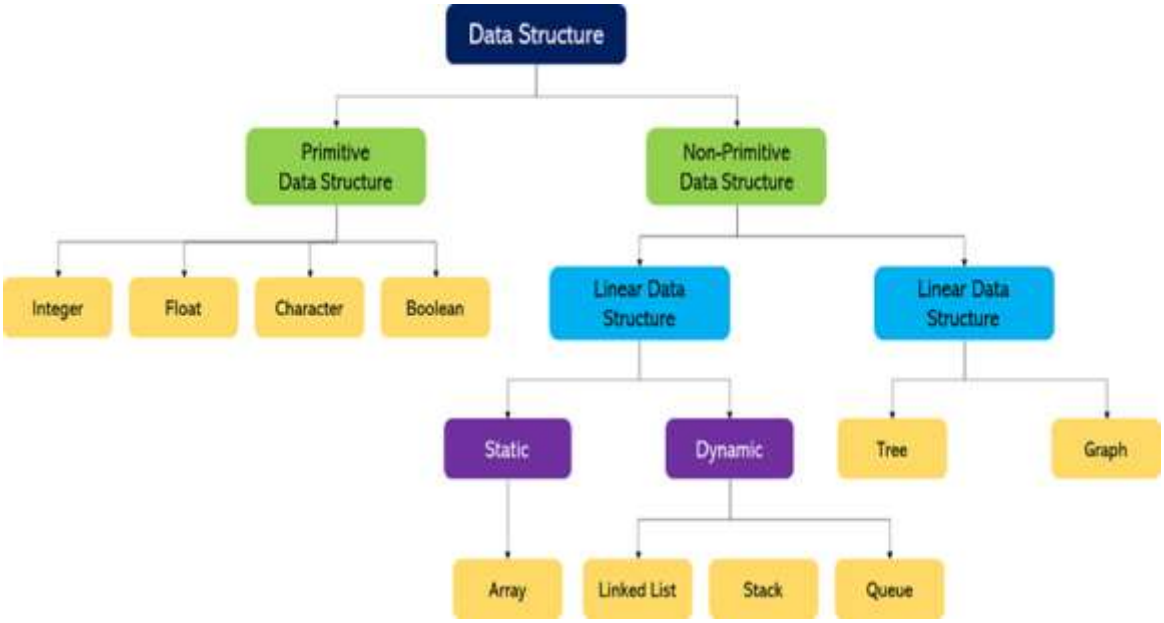
Some of the important features of data structures are:

- **Robustness:** Generally, all computer programmers wish to produce software that generates correct output for every possible input provided to it, as well as execute efficiently on all hardware platforms. This kind of robust software must be able to manage both valid and invalid inputs.
- **Adaptability:** Developing software projects such as word processors, Web browsers and Internet search engine involves large software systems that work or execute correctly and efficiently for many years. Moreover,

software evolves due to ever changing market conditions or due to emerging technologies.

- **Reusability:** Reusability and adaptability go hand-in-hand.It is a known fact that the programmer requires many resources for developing any software, which makes it an expensive enterprise. However, if the software is developed in a reusable and adaptable way, then it can be implemented in most of the future applications. Thus, by implementing quality data structures, it is possible to develop reusable software, which tends to be cost effective and time saving.

<u>Classification of Data Structures</u>:



1. **Primitive data structures:**

- The primitive data structures are integers, floating point numbers, characters, string constants, pointers etc.

int a =10;

2. **Non-primitive data structures:**

- It is advanced data structure emphasizing on structuring of a group of data items. They cannot be directly operated upon by the machine instructions.

Example: Array, list, files, linked list, trees and graphs fall in this category.

2.1 Linear and non-linear data structures:

- In a linear data structure, the data items are arranged in a linear sequence.

For example: array.

- In a non-linear data structure, the data items are not in sequence.

For Example: trees and graphs.

3. Homogeneous and non-homogeneous data structures:

- In homogeneous data structure, all the elements are of same type.

For Example: arrays.

- In non-homogeneous data structure, the elements may or may not be of the same type.

For Example: Records.

4. Static and dynamic data structures:

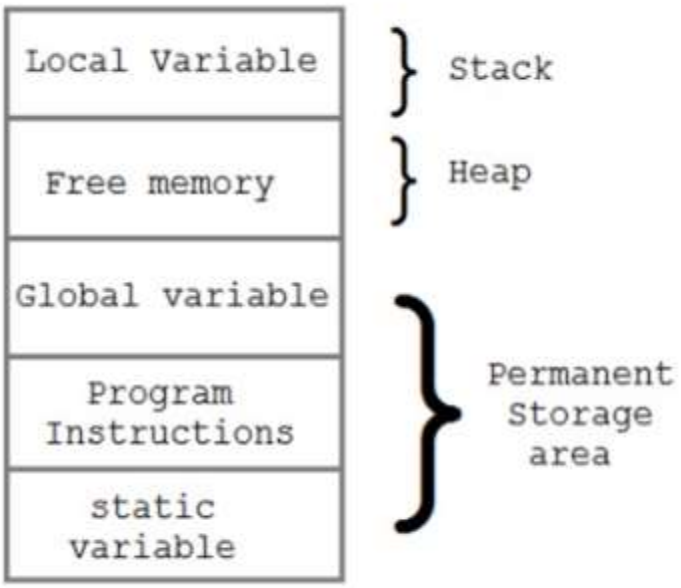
- In Static data structure the size of the structure is fixed.
- The content of the data structure can be modified but without changing the memory space allocated to it.

Example: Array

- In Dynamic data structure the size of the structure is not fixed and can be modified during the operations performed on it.

- Dynamic data structures are designed to facilitate change of data structures in the run time.

Example: Linked List



Differences between static and dynamic memory allocation

Static Memory Allocation	Dynamic Memory Allocation
Constant (Invariable) memory is reserved at compile-time of our program that can't be modified.	Dynamic (Variable) memory is reserved at run-time of our program that can be modified.
It is used at compile-time of our program and is also known as compile-time memory allocation.	It is used at run-time of our program and is also known as run-time memory allocation.
We can't allocate or deallocate a memory block during run-time.	We can allocate and deallocate a memory block during run-time.
Stack space is used in Static Memory Allocation.	Heap space is used in Dynamic Memory Allocation.
It doesn't provide reusability of memory, while the program is running. So, it is less efficient.	It provides reusability of memory, while the program is running. So, it is more efficient.

Memory allocation functions

- There are 4 library functions provided by C defined under <stdlib.h> header file to facilitate dynamic memory allocation in C programming.
- They are:
    - malloc()
    - calloc()
    - free()
    - realloc()

malloc():

- “malloc” or “memory allocation” method in C is used to dynamically allocate a single large block of memory with the specified size.
- It returns a pointer of type void which can be cast into a pointer of any form.

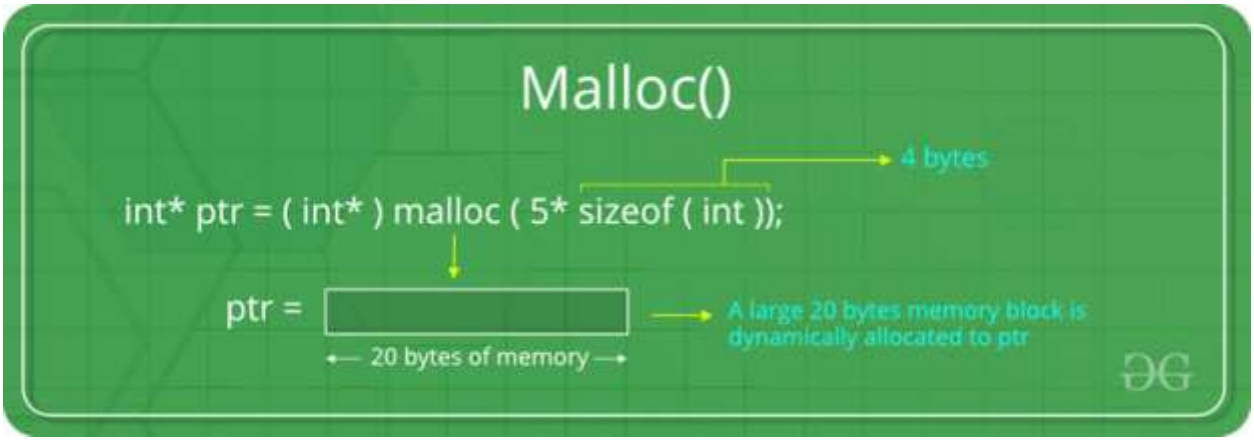
Syntax:

```
ptr = (cast-type*) malloc(byte-size)
```

For Example:

```
ptr = (int*) malloc(100 * sizeof(int));
```

- Since the size of int is 4 bytes, this statement will allocate 400 bytes of memory. And, the pointer ptr holds the address of the first byte in the allocated memory.



malloc(): example

```
#include <stdio.h>

#include <stdlib.h>

int main()

{

        int* ptr;

        int n=5, i, sum = 0;

        ptr = (int*)malloc(n * sizeof(int));

   for (i = 0; i < n; ++i) {

      ptr[i] = i + 1;

   }

   printf("The elements of the array are: ");

   for (i = 0; i < n; ++i) {

      printf("%d, ", ptr[i]);

   }

        return 0;      }
```

calloc()

- “calloc” or “contiguous allocation” method in C is used to dynamically allocate the specified number of blocks of memory of the specified type.
- It also returns a pointer of type void which can be cast into a pointer of any form.
- It initializes each block with a default value ‘0’.

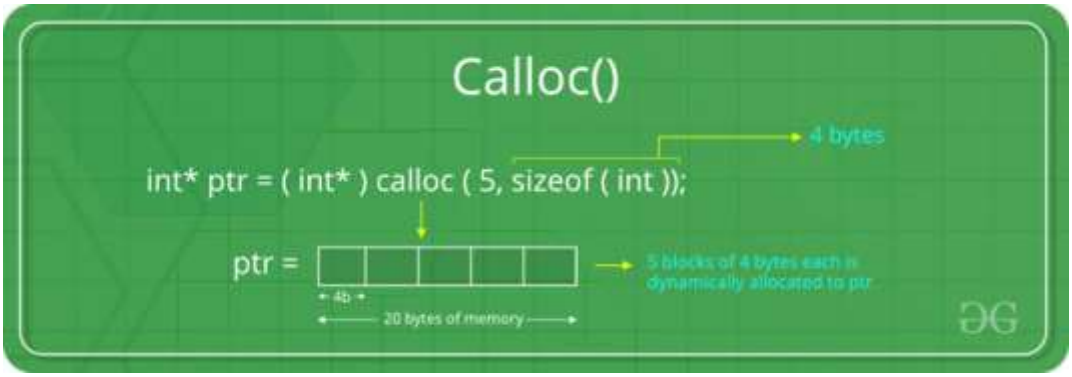
Syntax:

```
ptr = (cast-type*)calloc(n, element-size);
```

For Example:

```
ptr = (float*) calloc(25, sizeof(float));
```

This statement allocates contiguous space in memory for 25 elements each with the size of the float.




```
#include <stdio.h>

#include <stdlib.h>

 nt main()

{

        // This pointer will hold the
```

```
// base address of the block created
int* ptr;
int n, i;
 // Get the number of elements for the array
n = 5;
printf("Enter number of elements: %d\n", n);

// Dynamically allocate memory using calloc()
ptr = (int*)calloc(n, sizeof(int));
 // Check if the memory has been successfully
// allocated by calloc or not
if (ptr == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
}
else {
         // Memory has been successfully allocated
        printf("Memory successfully allocated using calloc.\n");
         // Get the elements of the array
        for (i = 0; i < n; ++i) {
                ptr[i] = i + 1;
```

```
        }

        // Print the elements of the array
        printf("The elements of the array are: ");
        for (i = 0; i < n; ++i) {
            printf("%d, ", ptr[i]);
        }
    }
    return 0;
}
```

Output:

Enter number of elements: 5

Memory successfully allocated using calloc.

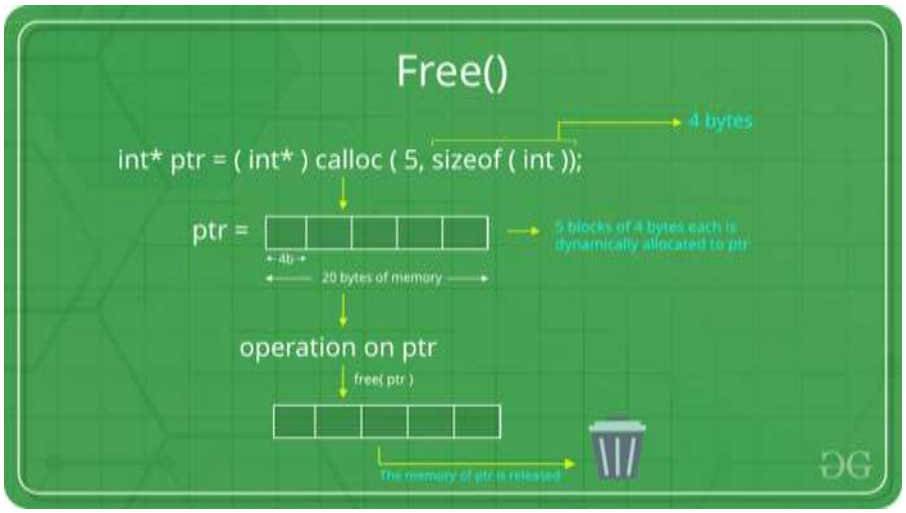
The elements of the array are: 1, 2, 3, 4, 5,

free() method

- **"free"** method in C is used to dynamically **de-allocate** the memory.
- The memory allocated using functions malloc() and calloc() is not de-allocated on their own.
- Hence the free() method is used, whenever the dynamic memory allocation takes place.
- It helps to reduce wastage of memory by freeing it.

Syntax of free() in C

```
free(ptr);
```



Example of free() in C

```
#include <stdio.h>

#include <stdlib.h>

 int main()

{

    // This pointer will hold the

   // base address of the block created

   int *ptr, *ptr1;

   int n, i;

    // Get the number of elements for the array

   n = 5;

   printf("Enter number of elements: %d\n", n);

    // Dynamically allocate memory using malloc()

   ptr = (int*)malloc(n * sizeof(int));

    // Dynamically allocate memory using calloc()
```

```
ptr1 = (int*)calloc(n, sizeof(int));
 // Check if the memory has been successfully
// allocated by malloc or not
if (ptr == NULL || ptr1 == NULL) {
    printf("Memory not allocated.\n");
    exit(0);
}
else {


    // Memory has been successfully allocated
    printf("Memory successfully allocated using malloc.\n");
     // Free the memory
    free(ptr);
    printf("Malloc Memory successfully freed.\n");
     // Memory has been successfully allocated
    printf("\nMemory successfully allocated using calloc.\n");
     // Free the memory
    free(ptr1);
    printf("Calloc Memory successfully freed.\n");
}
 return 0;
```

}

Output

Enter number of elements: 5
Memory successfully allocated using malloc.
Malloc Memory successfully freed.
Memory successfully allocated using calloc.
Calloc Memory successfully freed.

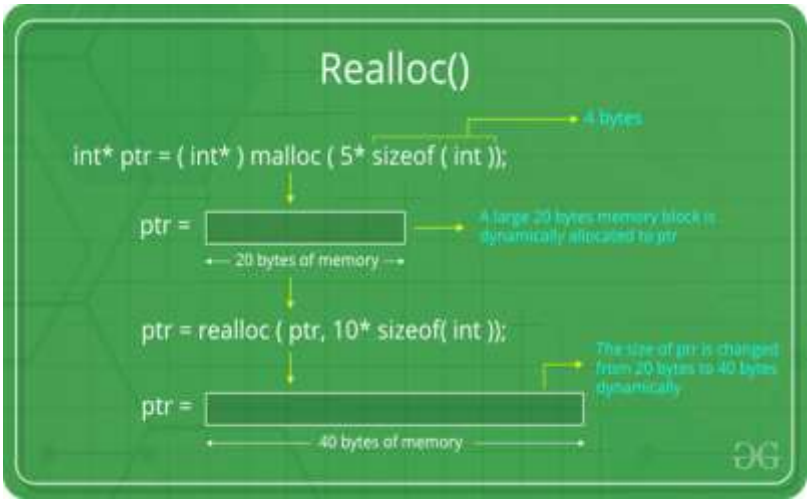
realloc() method

- **“realloc”** or **“re-allocation”** method in C is used to dynamically change the memory allocation of a previously allocated memory.
- In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to **dynamically re-allocate memory**.
- re-allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.

Syntax of realloc() in C

ptr = realloc(ptr, newSize);

where ptr is reallocated with new size 'newSize'.



- If space is insufficient, allocation fails and returns a NULL pointer.

Example of realloc() in C

```
#include <stdio.h>

#include <stdlib.h>

 int main()

{

     // This pointer will hold the

    // base address of the block created

    int* ptr;

    int n, i;

     // Get the number of elements for the array

    n = 5;

    printf("Enter number of elements: %d\n", n);


    // Dynamically allocate memory using calloc()

    ptr = (int*)calloc(n, sizeof(int));

     // Check if the memory has been successfully

    // allocated by malloc or not

    if (ptr == NULL) {

        printf("Memory not allocated.\n");

        exit(0);

    }
```

```
else {
    // Memory has been successfully allocated
   printf("Memory successfully allocated using calloc.\n");
    // Get the elements of the array
   for (i = 0; i < n; ++i) {
       ptr[i] = i + 1;
   }
    // Print the elements of the array
   printf("The elements of the array are: ");
   for (i = 0; i < n; ++i) {
       printf("%d, ", ptr[i]);
   }
    // Get the new size for the array
   n = 10;
   printf("\n\nEnter the new size of the array: %d\n", n);
    // Dynamically re-allocate memory using realloc()
   ptr = (int*)realloc(ptr, n * sizeof(int));
    // Memory has been successfully allocated
   printf("Memory successfully re-allocated using realloc.\n");


   // Get the new elements of the array
```

```
        for (i = 5; i < n; ++i) {
            ptr[i] = i + 1;
        }
         // Print the elements of the array
        printf("The elements of the array are: ");
        for (i = 0; i < n; ++i) {
            printf("%d, ", ptr[i]);
        }
         free(ptr);
    }
    return 0;
}
```

Output

Enter number of elements: 5

Memory successfully allocated using calloc.

The elements of the array are: 1, 2, 3, 4, 5,

Enter the new size of the array: 10

Memory successfully re-allocated using realloc.

The elements of the array are: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,

Recursion

Definition

- Recursion is the process of function call itself or repeating items in a self-similar way.
- In programming languages, if a program allows you to call a function inside the same function, then it is called a recursive call of the function.

```
void recursion() {
  recursion(); /* function calls itself */
}

int main() {
  recursion();
}
```

- The C programming language supports recursion, i.e., a function to call itself.
- Recursive functions are very useful to solve many mathematical problems, such as calculating the factorial of a number, generating Fibonacci series, etc.

```
int fact(int n)
{
   if (n < = 1) // base case
      return 1;
   else
      return n*fact(n-1);
}
```

Example: Sum of Natural Numbers Using Recursion

```
#include <stdio.h>

int sum(int n);

int main() {

   int number, result;
```

```
    printf("Enter a positive integer: ");
    scanf("%d", &number);
    result = sum(number);
    printf("sum = %d", result);
    return 0;
}
int sum(int n) {
    if (n != 0)
        // sum() function calls itself
        return n + sum(n-1);
    else
        return n;
}
```

Output

Enter a positive integer:3

sum = 6

GCD (Greatest Common Divisor) or HCF (Highest Common Factor) of two numbers is the largest number that divides both of them.

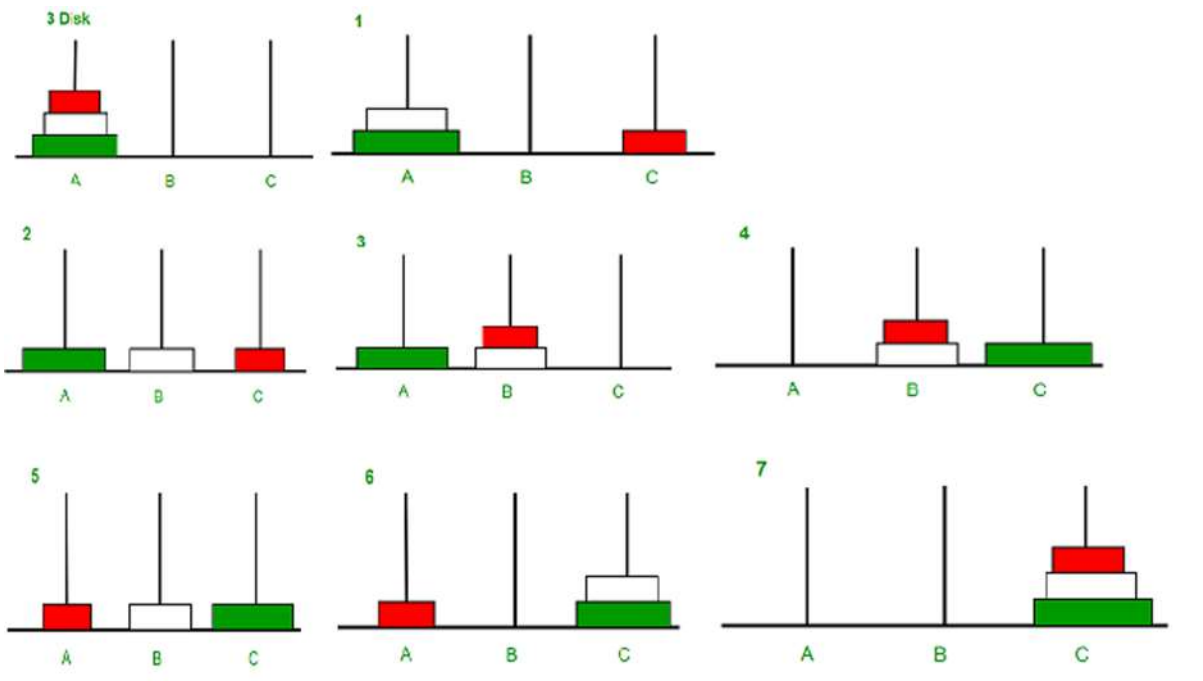
```
#include<stdio.h>
 void main()
{
            int a,b,res;
```

```
        clrscr();
        printf("\nGreatest Common Divisor\n");
        printf("Enter the value for a : ");
        scanf("%d",&a);
        printf("Enter the value for b : ");
        scanf("%d",&b);
        res=gcd(a,b);
        printf("The Greatest Common Divisor of %d and %d is %d",a,b,res);
        getch();
}
 int gcd(int a,int b)
{
        if(a==b)
        {
                return(a);
        }
        else
        {
                if(a>b)
                        return(gcd(a-b,b));
                else
```

```
                    return(gcd(a,b-a));
        }
}
```

Tower of Hanoi

- Tower of Hanoi is a mathematical puzzle where we have three rods and n disks.
- The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules:
  1) Only one disk can be moved at a time.
  2) Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack.



Algorithm:

Step 1 − Move n-1 disks from source to aux

Step 2 − Move nth disk from source to dest

Step 3 − Move n-1 disks from aux to dest

Code:

```
#include<stdio.h>

#include<math.h>

void main()

{

        int n;

        char A,B,C;

        void towers(int,char,char,char);

        clrscr();

        printf("\nTowers of Hanoi\n");

        printf("\nEnter the number of disks : ");

        scanf("%d",&n);

        printf("The number of moves=%0.f\n",(pow(2,n)-1));

        printf("\nTowers of Hanoi simulation for %d disks\n",n);

        towers(n,'A','C','B');

        getch();

}

void towers(int n,char source,char dest,char aux)

{
```

```
        if(n==1)
        {
                printf("\nMove disk %d from %c to %c",n,source,dest);
                return;
        }
        towers(n-1,source,aux,dest);
        printf("\nMove disk %d from %c to %c",n,source,dest);
        towers(n-1,aux,dest,source);
}
```

Output

Towers of Hanoi

Enter the number of disks : 3

The number of moves=7

Towers of Hanoi simulation for 3 disks

Move disk 1 from A to C

Move disk 2 from A to B

Move disk 1 from C to B

Move disk 3 from A to C

Move disk 1 from B to A

Move disk 2 from B to C

Move disk 1 from A to C